

PROJECT REPORT

“Smart Inventory & Warehouse Management System”

A Database-Driven Web Application for Inventory, Stock, Warehouse, Analytics and Reorder Management

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

S Svara Haasini :2520090170

B Sudharshini :2520090227

Course: Database Systems Engineering & Distributed Backend Development (25CS1302E)

Academic Year: 2026

Under the Esteemed Guidance of

Dr M Gayatri

Course Faculty Guide

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

We hereby declare that the project report entitled **“Smart Inventory & Warehouse Management System”** is a record of the work carried out as part of the academic requirements of the **Bachelor of Technology** programme. The project was developed to demonstrate practical application of database management, backend development, web technologies, transaction processing, analytics and warehouse-oriented decision support.

We further declare that the work presented in this report is based on the implementation and testing performed for the project. Wherever external technologies, documentation or academic references have been used, they have been acknowledged appropriately.

The report has been prepared for academic evaluation and reflects the implemented functionality of the project together with clearly identified future enhancements.

Name	Roll Number	Signature
S Svara Haasini	2520090170	_____
B Sudharshini	2520090227	_____

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled **“Smart Inventory & Warehouse Management System”** is a bonafide record of the work carried out by **S Svara Haasini** (2520090170) and **B Sudharshini** (2520090227) as part of the academic requirements of the **Bachelor of Technology** programme in Computer Science and Information Technology.

The project demonstrates the application of relational database design, web application development, transaction management, inventory analytics and warehouse route optimization. The work has been completed under the guidance and supervision of the concerned course faculty.

Guide / Faculty

Signature: _____

Name: _____

Course Coordinator

Signature: _____

Name: _____

Head of Department

Signature: _____

Name: _____

ACKNOWLEDGEMENT

The successful completion of this project was possible because of the guidance, encouragement and support received throughout the development process. We sincerely thank our course faculty and mentors for providing the direction required to understand the problem, design the database, implement the web application and validate the final system.

We are grateful to the Department of Computer Science and Information Technology for providing the academic environment and resources needed to complete this project. The project gave us an opportunity to connect classroom concepts such as normalization, keys, joins, transactions, views, stored procedures and triggers with a working application.

We also thank our friends, classmates and family members for their encouragement and support during development, debugging, testing and documentation. Finally, we acknowledge everyone who contributed directly or indirectly to the completion of this project.

S SVARA HAASINI – 2520090170

B SUDHARSHINI – 2520090227

Abstract

The Smart Inventory & Warehouse Management System is a database-driven web application designed to improve the way products, suppliers, warehouses, stock levels and inventory transactions are managed. Traditional spreadsheet-based inventory management often becomes difficult to maintain as the number of products and stock movements increases. Manual updates can lead to inconsistent quantities, repeated data entry, delayed low-stock identification and difficulty in tracing previous transactions.

The proposed system provides a centralized interface through which users can manage product and category information, supplier details, warehouse records, stock levels and transaction history. MySQL provides the relational data layer, while the FastAPI backend handles authentication, validation, business logic, database communication and analytics endpoints. The browser-based frontend provides dashboards, forms, reports and notifications so that inventory operations can be performed without directly writing SQL.

The system also extends beyond basic CRUD functionality. It includes ABC inventory classification based on consumption value, demand-based reorder-point analysis using recent stock-out history, trigger-based reorder alerts, warehouse storage-location support and a Dijkstra-based picking-route optimizer. The ABC module classifies products using 70% and 90% cumulative consumption-value thresholds. The reorder-point module analyzes the most recent 30 days of OUT transactions, treats missing-demand days as zero demand and combines average daily demand, supplier lead time and safety stock to identify replenishment requirements.

The warehouse routing module models storage locations as a weighted graph and uses Dijkstra's shortest-path algorithm together with a nearest-neighbour strategy for multi-stop picking. Supporting tables such as storage locations, warehouse paths, product-location mappings and reorder alerts extend the core inventory schema. The result is a practical academic project that combines database engineering with application development and decision-support features.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement Abstract	
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Existing System 1.4 Proposed System 1.5 Objectives 1.6 Scope 1.7 Importance of the Application 1.8 Target Users	8-10
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Functional Requirements 2.4 Non-Functional Requirements	11-12
3	Technology Stack	3.1 Frontend Technologies 3.2 Backend Technologies 3.3 Database Technologies 3.4 Authentication and Security 3.5 Development Tools	13-14
4	System Architecture	4.1 High-Level Architecture 4.2 Layer Description 4.3 Data Flow 4.4 Deployment Architecture	15-16
5	Design	5.1 Database Design 5.2 ER Model 5.3 Core Relationships 5.4 Supporting Tables 5.5 Database Views, Procedures and Triggers	17-19
6	Implementation	6.1 Module-Wise Implementation 6.2 Frontend Logic 6.3 API Integration 6.4 Authentication and User Isolation 6.5 Inventory Transactions	20-21
7	Smart Features and Algorithms	7.1 ABC Classification 7.2 Demand-Based Reorder Point 7.3 Trigger-Based Reorder Alerts 7.4 Dijkstra Picking Route Optimization 7.5 ABC-Based Storage Assignment 7.6 CSV Import and Export	22-23

S. No.	Chapters	Topics	Page.no
8	Testing	8.1 Testing Strategy 8.2 API Testing 8.3 Integration Testing 8.4 Algorithm Testing 8.5 Test Cases and Results	24-25
9	Deployment	9.1 Local Deployment 9.2 Environment Configuration 9.3 Production Considerations	26
10	Challenges and Limitations	10.1 Development Challenges 10.2 Solutions Applied 10.3 Current Limitations	27
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations and Optimizations	28
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	29-30
13	References	References	31
14	Appendices	Appendix A – Application Screenshots Appendix B – Representative API Endpoints Appendix C – Key Formulas Appendix D – User Workflow Appendix E – Documentation Accuracy Note	32-40

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Inventory is one of the most important operational resources for organizations that purchase, store, move and sell physical products. When inventory is distributed across multiple warehouses, the problem becomes more than simply counting items. Staff need to know what is available, where it is stored, which supplier provides it, how stock has moved over time and which products require attention.

In small and medium-sized operations, inventory records are often maintained using spreadsheets, paper registers or disconnected software. These approaches can work for a small number of products, but they become difficult to maintain as transaction volume grows. Repeated data entry, accidental overwriting, inconsistent stock counts and delayed reporting can affect day-to-day decisions.

The Smart Inventory & Warehouse Management System was developed to address these practical issues through a centralized relational database and a web-based interface. Instead of requiring staff to manipulate tables directly, the application provides structured screens for product management, suppliers, warehouses, stock operations, analytics and alerts.

The project also explores how database systems can support decision-making rather than only storing records. Inventory analytics are used to classify products through ABC analysis and identify replenishment needs through demand-based reorder-point calculation. Warehouse locations are represented as a graph so that picking paths can be optimized using shortest-path computation.

1.2 Problem Statement

The main problem is the difficulty of maintaining accurate, timely and useful inventory information when stock is distributed across products, suppliers and warehouse locations. Manual systems make it difficult to maintain relationships between records, trace stock movements and identify low-stock conditions early.

A second problem is that basic inventory applications often stop at CRUD operations. They can store product information but provide limited support for deciding which products are important, when replenishment may be required or how warehouse staff can reduce unnecessary walking during picking.

1.3 Existing System

- Spreadsheet or paper-based inventory records with limited automation.
- Manual stock updates after receiving, dispatching or transferring products.
- Repeated entry of product, supplier and warehouse information.
- Limited relationship enforcement between business entities.
- Low-stock checks performed manually or through simple threshold lists.

- Reports prepared manually from raw records.
- Little or no support for inventory classification or warehouse route optimization.

1.4 Proposed System

The proposed system replaces fragmented record keeping with a centralized MySQL-backed web application. The backend exposes REST-style endpoints through FastAPI and the frontend provides an accessible browser interface. Database constraints, stored procedures and triggers help keep operational data consistent, while analytics endpoints provide decision-support features.

1.5 Objectives

1. Create a centralized inventory database for products, categories, suppliers, warehouses, stock and transactions.
2. Provide secure authentication and isolate each user's inventory data using the authenticated user identifier.
3. Support stock-in, stock-out and warehouse transfer operations with validation and transaction-safe updates.
4. Automatically identify reorder conditions and expose them through the application.
5. Classify products using consumption-value based ABC analysis.
6. Calculate demand-based reorder points using recent OUT transaction history.
7. Model warehouse storage locations as a graph and optimize picking paths.
8. Provide CSV import/export facilities for practical data handling.
9. Generate useful inventory summaries through SQL views, joins and aggregates.

1.6 Scope

The scope covers the complete lifecycle of inventory information from master-data creation to stock movement, monitoring, analytics and reporting. It includes product and category management, supplier and warehouse management, stock records, transaction history, reorder alerts, analytics and warehouse route support.

The current system is primarily a web-based academic prototype. Production-scale deployment, advanced forecasting, barcode hardware, automated procurement and enterprise-grade role management are considered future extensions rather than claims of the current implementation.

1.7 Importance of the Application

The application is important because it connects core database concepts to a realistic business problem. A well-designed relational schema reduces redundancy and improves consistency,

while controlled transaction operations protect stock data. Analytics features demonstrate how the same database can support operational decisions instead of being used only as a storage layer.

1.8 Target Users

- Warehouse staff and inventory operators
- Small and medium-sized businesses
- Store or warehouse supervisors
- Inventory administrators
- Students and developers studying database-backed applications

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Component	Recommended Development Requirement	Purpose
Processor	Intel Core i5 / AMD Ryzen 5 or equivalent	Runs IDE, browser, database and backend simultaneously
RAM	8 GB minimum; 16 GB preferred	Supports development tools and local services
Storage	SSD with at least 20–30 GB free project space	Fast application and database access
Display	1366×768 or higher	Frontend development and testing
Network	Stable internet connection	Package installation, documentation and optional deployment
Client device	Desktop/laptop/tablet with modern browser	Accesses the web application

2.2 Software Requirements

Software	Role
Windows / Linux / macOS	Development operating system
Python 3.x	Backend runtime
FastAPI	REST API framework
Uvicorn	ASGI development server
MySQL 8.0+	Relational database
PyMySQL	Python–MySQL connectivity
python-jose	JWT-based authentication support
HTML, CSS, JavaScript	Frontend
Bootstrap / responsive CSS	User interface and responsive layout
VS Code	Development environment
Git / GitHub	Version control and project collaboration
Modern browser	Frontend and API testing

2.3 Functional Requirements

- User registration/login and authenticated access.
- Product, category, supplier and warehouse management.
- Stock viewing by product and warehouse.
- Stock-in, stock-out and transfer operations.
- Transaction-history retrieval.
- Low-stock/reorder alert monitoring.
- ABC classification and analytics.
- Demand-based reorder-point calculation.
- Warehouse path and picking-route calculation.
- CSV product import and inventory export.
- Dashboard summaries and reports.

2.4 Non-Functional Requirements

Requirement	System Approach
Usability	Simple browser interface with forms, dashboards, tables and notifications
Reliability	Database constraints, validation and transaction-controlled operations
Security	JWT authentication and user-scoped data access
Performance	Indexed relational queries, SQL aggregation and reusable views
Maintainability	Modular FastAPI endpoints and separated frontend/backend/database layers
Scalability	Cloud-suitable architecture with production configuration changes
Portability	Runs in a standard Python/MySQL development environment

CHAPTER -3 TECHNOLOGY STACK

3.1 Frontend Technologies

The frontend provides the visual layer through which users interact with the inventory system. HTML structures the pages, CSS controls layout and presentation, JavaScript manages dynamic interaction and API communication, and responsive styling allows the application to remain usable across desktop and smaller screens.

The interface includes inventory dashboards, product forms, supplier and warehouse views, transaction screens, analytics results and reorder notifications. The goal is to keep database operations understandable to non-technical users.

3.2 Backend Technologies

The backend is implemented using Python and FastAPI. FastAPI provides structured routing, request validation and JSON-based API responses. Uvicorn serves the application during local development. The backend contains authentication logic, product and warehouse operations, transaction handling, analytics and route-optimization endpoints.

3.3 Database Technologies

MySQL is the central relational database. The schema uses primary keys and foreign keys to represent relationships between products, categories, suppliers, warehouses, stock records and transactions. Supporting tables extend the database for storage locations, warehouse paths, product-location mappings and reorder alerts.

SQL joins and aggregate functions support reports and analytics. Views provide reusable reporting logic, while stored procedures encapsulate important stock operations. MySQL triggers automate reorder-alert creation when configured stock thresholds are reached.

3.4 Authentication and Security

The application uses JWT-based authentication. After successful login, the authenticated user's identity is available to protected endpoints. Inventory queries and operations are scoped using the user identifier so that one user's records are not unintentionally exposed to another user.

The current development configuration supports environment variables for database settings and the JWT signing secret. Local fallback values are present for development convenience; production deployment should replace these with secure environment configuration and restrict CORS to the deployed frontend origin.

3.5 Development Tools

Tool	Use
Visual Studio Code	Coding and project management

Tool	Use
MySQL Workbench / MySQL tools	Database creation, queries and inspection
Browser Developer Tools	Frontend debugging and responsive testing
Uvicorn	Local FastAPI server
Git / GitHub	Version control
Postman / API client	Endpoint validation and request testing

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-Level Architecture

The system follows a layered architecture in which the browser communicates with the FastAPI application, the application communicates with MySQL, and analytics modules operate on validated database data. This separation makes the project easier to understand, test and extend.

Smart Inventory & Warehouse Management System – System Architecture

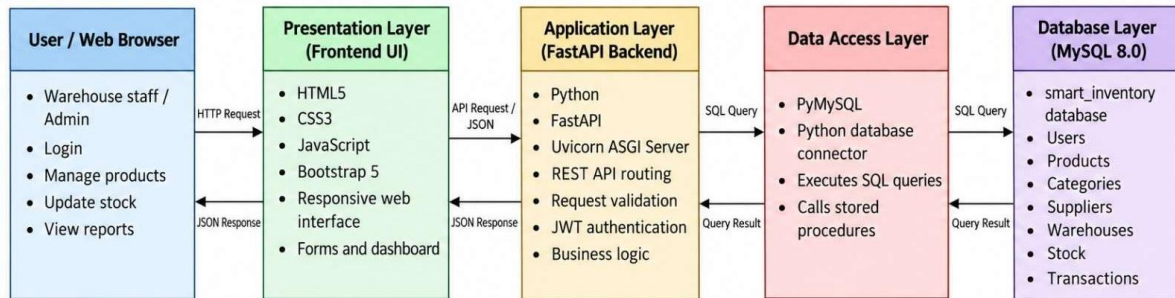


Figure 4.1 – System Architecture of the Smart Inventory & Warehouse Management System

Layer	Main Components	Responsibility
Presentation Layer	HTML, CSS, JavaScript, responsive UI	Forms, dashboards, tables, notifications and user interaction
Application Layer	FastAPI, Python modules	Routing, validation, authentication, business logic and analytics
Data Access Layer	PyMySQL / SQL queries	Connection management, parameterized queries and stored-procedure calls
Database Layer	MySQL	Persistent data, relationships, views, procedures and triggers
Analytics Layer	Python algorithms + SQL data	ABC, ROP and warehouse route optimization

4.2 Layer Description

A typical request begins in the browser. The frontend sends a request to a protected or public API endpoint. FastAPI validates the request and, where required, verifies the JWT. The backend then performs a database query or invokes a stored procedure. MySQL returns the result, and the backend converts it into a JSON response for the frontend.

4.3 Data Flow

User action → Frontend request → FastAPI route → authentication/validation → SQL query or stored procedure → MySQL → processed response → frontend update.

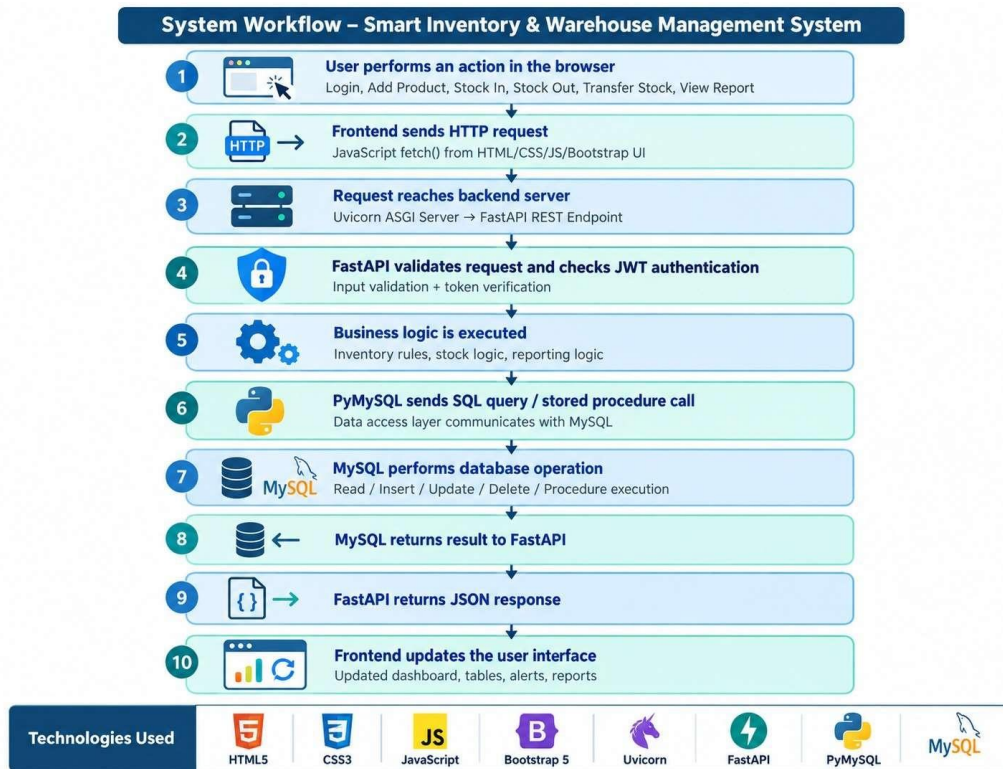


Figure 4.2 – System Workflow of the Smart Inventory & Warehouse Management System

4.4 Deployment Architecture

The application is currently suitable for local development using a Python virtual environment, Uvicorn and a local MySQL instance. For production, the frontend and backend can be deployed as separate services and the database can be hosted on a managed MySQL service. Minor configuration changes are required, including updating the API base URL, supplying secure environment variables and restricting CORS.

CHAPTER -5 DESIGN

5.1 Database Design

The database is organized around core business entities and supporting tables used by analytics and automation. The design separates master data from transactional data and uses relationships to avoid unnecessary duplication.

5.2 ER Model

The seven core business entities are Product, Category, Supplier, Warehouse, Stock, Transaction and User. Stock acts as the relationship between products and warehouses, allowing the same product to exist at multiple warehouse locations. Transactions record movement history such as stock-in, stock-out and transfer operations.

Entity	Important Role
User	Identifies the owner/user scope of inventory records
Category	Groups products into meaningful categories
Supplier	Stores supplier information associated with products
Product	Stores product master information, price and inventory configuration
Warehouse	Stores warehouse details and capacity
Stock	Represents product quantity held at a warehouse
Transaction	Records inventory movements over time

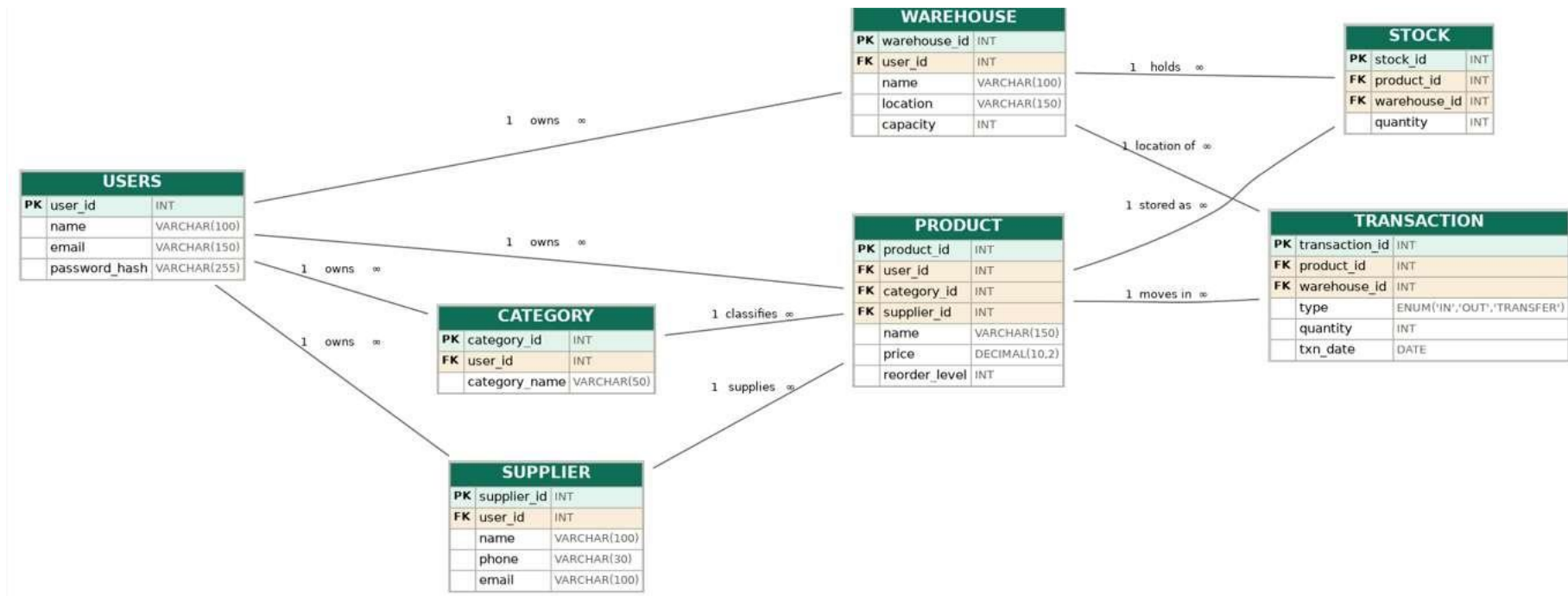


Figure 5.1 – ER Diagram of the Smart Inventory & Warehouse Management System

5.3 Core Relationships

- One category can be associated with many products.
- One supplier can supply many products.
- One warehouse can contain many stock records.
- One product can appear in stock records for multiple warehouses.
- One product can have many transaction records.
- User ownership is applied to relevant inventory records through user-scoped identifiers.

5.4 Supporting Tables

Supporting Table	Purpose
storage_location	Represents physical warehouse nodes/bins used by route optimization.
warehouse_path	Stores connections between storage locations and their distances.
product_location	Maps products to their warehouse storage locations.
reorder_alert	Stores alert records created when stock reaches a reorder condition.

The core schema therefore contains seven principal business entities plus four supporting tables used for analytics, physical storage modelling and automated alerts.

5.5 Database Views, Procedures and Triggers

The database includes reusable views for inventory reporting and summary information. Stored procedures are used for important operational actions, while triggers provide automatic alert generation.

Database Object	Purpose
sp_stock_in	Validates and records incoming stock while updating stock quantities
sp_stock_out	Checks availability and records outgoing stock
sp_transfer	Validates warehouse-to-warehouse stock movement
Reorder triggers	Create reorder-alert records when configured thresholds are reached
Reporting views	Provide reusable inventory, utilization, value and catalog summaries

CHAPTER -6 IMPLEMENTATION

6.1 Module-Wise Implementation

Module 1 – Authentication and User Management

The authentication module handles user registration and login. Passwords are processed before storage and successful login produces a JWT token. Protected API endpoints read the authenticated identity and use it to scope inventory operations.

Module 2 – Product and Category Management

Users can create, view, update and delete product/category information through frontend forms connected to FastAPI endpoints. Product information can include name, price, category, supplier and inventory-related configuration such as reorder level and lead-time information.

Module 3 – Supplier and Warehouse Management

Supplier records store contact and business information. Warehouse records maintain warehouse identity, location and capacity information. These entities provide the master data required for stock and transaction operations.

Module 4 – Stock and Transaction Management

Stock-in, stock-out and transfer are the core operational workflows. Validation is performed before quantities are changed. The database records transaction history so users can trace inventory movement rather than relying only on the current stock balance.

Module 5 – Analytics and Reports

The analytics module exposes inventory summaries, consumption-based classification and reorder-point analysis. Reports can also be exported for offline use.

Module 6 – Warehouse Route Optimization

The warehouse routing module uses storage locations as graph nodes and warehouse paths as weighted edges. The system calculates shortest paths between locations and builds a multi-stop picking route using a nearest-neighbour selection strategy.

6.2 Frontend Logic

The frontend is organized around user workflows rather than database tables alone. Forms collect valid inputs, dashboards summarize important information, tables display records and notification elements expose reorder conditions. JavaScript handles asynchronous requests so that data can be refreshed without repeatedly reloading the entire interface.

6.3 API Integration

FastAPI endpoints connect the frontend to the database and analytics logic. The API layer provides a consistent boundary between user interaction and database operations. Representative endpoint groups include authentication, products, inventory, transactions, analytics, route optimization and reorder alerts.

6.4 Authentication and User Isolation

Protected requests include the user's JWT. The backend decodes the token, obtains the user identity and applies that identity when retrieving or modifying inventory data. This prevents the application from treating all inventory records as globally accessible.

6.5 Inventory Transactions

The three primary database procedures are stock-in, stock-out and transfer. Stock-in increases available quantity after validation. Stock-out decreases quantity only when sufficient stock is available. Transfer moves quantity between warehouses while validating the source and destination conditions. The use of database-side procedures helps centralize important operational rules.

CHAPTER -7 SMART FEATURES AND ALGORITHMS

7.1 ABC Classification

ABC analysis prioritizes products according to their contribution to inventory consumption value. The implementation calculates consumption value from the total quantity recorded in OUT transactions multiplied by the product unit price.

Products are sorted from highest to lowest consumption value. The current implementation uses cumulative-value thresholds of 70% and 90%. Products contributing to the first 70% are classified as A, products taking the cumulative contribution beyond 70% and up to 90% are classified as B, and the remaining products are classified as C.

The classification is performed in the FastAPI/Python backend after SQL retrieves the required transaction aggregates. Therefore, the ABC algorithm itself is not described as a MySQL window-function implementation.

7.2 Demand-Based Reorder Point

The reorder-point algorithm analyzes the most recent 30 days of OUT transactions. Missing demand days are treated as zero-demand days so that the average represents the full 30-day period rather than only days on which a transaction occurred.

The core formula is:

$$\text{ROP} = \text{Average Daily Demand} \times \text{Supplier Lead Time} + \text{Safety Stock}$$

The calculated ROP is compared with the product's current total stock. If current stock is at or below the calculated reorder point, the system identifies the product as requiring replenishment. When safety stock is not explicitly supplied, the implementation can derive a buffer based on expected demand during the lead-time period.

This feature determines when replenishment attention is required. It does not claim to automatically calculate a purchase order quantity or place an order.

7.3 Trigger-Based Reorder Alerts

Database triggers monitor relevant stock changes. When stock reaches or falls below the configured reorder level, a corresponding record can be created in the `reorder_alert` table. This moves low-stock detection closer to the database operation itself and provides a persistent alert record for the application.

7.4 Dijkstra Picking Route Optimization

Warehouse locations are modelled as graph nodes and paths between locations are represented as weighted edges. Dijkstra's algorithm is used to calculate shortest paths between locations. For multiple products, a nearest-neighbour strategy chooses the next unvisited picking location, after which shortest-path distances are used to build the route.

The system can compare the optimized route with the original/requested sequence and report distance reduction. This is more accurately described as a combination of shortest-path computation and a heuristic multi-stop ordering strategy rather than claiming that Dijkstra alone solves the complete multi-stop routing problem.

7.5 ABC-Based Storage Assignment

The project also supports storage assignment logic in which higher-priority A-class products can be placed closer to the warehouse entrance. This connects inventory importance with physical warehouse organization and can reduce picking effort for frequently or economically important items.

7.6 CSV Import and Export

CSV support improves practical usability. Product data can be imported in bulk instead of being entered one record at a time, while inventory information can be exported for reporting, sharing or offline analysis.

CHAPTER -8 TESTING

8.1 Testing Strategy

Testing was approached at three levels: database-level validation, API-level testing and end-to-end browser verification. The goal was not only to confirm that pages load, but also to verify that a user action produces the expected database state and that analytics return meaningful results.

8.2 API Testing

FastAPI endpoints were tested using browser/API tools and the generated API documentation. Particular attention was given to authentication, product operations, inventory operations, analytics and reorder-point calculation. The reorder-point endpoint was verified after correcting its registration and handling MySQL Decimal values correctly in the backend.

8.3 Integration Testing

Integration testing follows the complete flow from frontend action to API request, database operation and frontend response. Example workflows include adding a product, recording stock, viewing the updated quantity, checking alerts and running analytics.

8.4 Algorithm Testing

Algorithm	Validation Focus	Expected Result
ABC Classification	Consumption value and cumulative thresholds	A/B/C class returned using 70% and 90% thresholds
ROP	30-day OUT history, zero-demand days, lead time and safety stock	Calculated ROP and reorder decision returned
Dijkstra Route	Storage graph, weighted paths and multiple stops	Shortest-path based optimized picking route returned
Reorder Trigger	Stock reaches reorder level	Alert record created

8.5 Test Cases and Results

Test ID	Test Case	Expected Result	Status
TC-01	Register/login with valid credentials	JWT/session created	Pass
TC-02	Add a valid product	Product stored and displayed	Pass
TC-03	Stock-in operation	Quantity increases and transaction recorded	Pass

Test ID	Test Case	Expected Result	Status
TC-04	Stock-out with sufficient stock	Quantity decreases and transaction recorded	Pass
TC-05	Stock-out beyond available quantity	Operation rejected	Pass
TC-06	Warehouse transfer	Source/destination quantities updated correctly	Pass
TC-07	Low-stock threshold reached	Reorder alert generated/displayed	Pass
TC-08	ABC analytics	Products classified using cumulative consumption value	Pass
TC-09	30-day ROP calculation	ROP returned without Decimal/float failure	Pass
TC-10	Route optimization	Shortest-path based picking route returned	Pass
TC-11	CSV product import	Valid rows processed	Pass
TC-12	User-scoped inventory access	Only authenticated user's data returned	Pass

The testing results indicate that the principal application workflows and the corrected analytics path are functioning as intended in the current project version. Performance figures are intentionally not represented as measured percentages unless a controlled benchmark has been recorded.

CHAPTER -9 DEPLOYMENT

9.1 Local Deployment

The application can be run locally using a Python virtual environment and a MySQL database. The backend dependencies are installed from the project requirements file. The database schema and supporting SQL objects are created in MySQL before starting the API server.

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
uvicorn main:app --reload
```

The frontend is then opened through the configured local application interface, and the API documentation can be used to inspect and test backend routes.

9.2 Environment Configuration

The backend supports environment-based configuration for database connectivity and authentication secrets. For local development, fallback values may be present to simplify setup. For a production deployment, database credentials and JWT secrets should be supplied through secure environment variables rather than committed to source control.

9.3 Production Considerations

- Update the frontend API base URL from localhost to the deployed backend address.
- Use a managed MySQL database or secured database server.
- Store secrets in a production secret-management mechanism.
- Restrict CORS to trusted frontend origins.
- Enable HTTPS and secure transport.
- Use process management and monitoring for the FastAPI service.
- Create database backups and test restoration procedures.

CHAPTER -10 CHALLENGES AND LIMITATIONS

10.1 Development Challenges

- Designing a relational schema that supports both basic inventory operations and warehouse analytics.
- Keeping stock quantities consistent when multiple types of transactions can modify them.
- Connecting database procedures and triggers with a web application without duplicating business rules unnecessarily.
- Handling numeric types returned by MySQL during analytics calculations.
- Representing physical warehouse paths as a graph suitable for shortest-path algorithms.
- Keeping the frontend simple while exposing advanced database functionality.

10.2 Solutions Applied

Normalization and foreign-key relationships were used to reduce duplication and protect referential integrity. Stock operations were centralized through stored procedures and validation logic. Trigger-based alerts were separated from application-level analytics. The reorder-point implementation was corrected to handle MySQL Decimal values consistently before performing arithmetic.

For route optimization, storage locations and paths were separated into supporting tables so that physical warehouse structure could be maintained independently from product master data.

10.3 Current Limitations

- The current system identifies when reordering is required but does not automatically generate purchase orders or calculate an optimized purchase quantity.
- Route optimization uses Dijkstra plus a nearest-neighbour heuristic; it is not an exact solution to every possible multi-stop warehouse routing problem.
- Advanced machine-learning demand forecasting is not part of the current implementation.
- Production deployment requires secure environment configuration, a production API base URL and appropriate CORS restrictions.
- Enterprise-grade role-based access control can be expanded beyond the current authenticated user isolation model.
- Real-time barcode/RFID hardware integration is not currently implemented.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

- Barcode and QR-code scanning for faster receiving and dispatch.
- RFID-based stock tracking for larger warehouse environments.
- Machine-learning demand forecasting using historical transaction patterns.
- Automated purchase-order generation after approval.
- Recommended reorder quantities based on demand, lead time, supplier constraints and budget.
- Advanced role-based access for administrators, warehouse managers and operators.
- Mobile-friendly warehouse picking interface.
- Real-time notifications through email or messaging services.
- More advanced route optimization for large numbers of picking stops.
- Cloud deployment with centralized monitoring and backups.

11.2 Possible Integrations and Optimizations

Future versions can integrate supplier APIs, barcode scanners, warehouse handheld devices and external reporting platforms. Analytics can be expanded from descriptive and rule-based methods toward forecasting models. The database can also be optimized further through carefully selected indexes, pagination, connection pooling and caching as the data volume grows.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

The Smart Inventory & Warehouse Management System demonstrates how a relational database and a modern web backend can transform routine inventory operations into a centralized and more reliable workflow. The project connects product, supplier, warehouse, stock and transaction information through a structured MySQL schema and exposes these capabilities through a FastAPI-powered web application.

The system goes beyond simple CRUD by adding inventory classification, demand-based replenishment analysis, automatic reorder alerts and warehouse route optimization. These features demonstrate the practical use of database systems, backend programming and algorithms within one coherent application.

12.2 What Was Achieved

- Centralized inventory and warehouse data management.
- Authenticated and user-scoped API access.
- Validated stock-in, stock-out and transfer operations.
- Transaction history for traceability.
- SQL views and aggregates for reporting.
- ABC classification using consumption-value thresholds.
- 30-day demand-based reorder-point analysis.
- Trigger-based reorder alerts.
- Dijkstra-based warehouse picking route optimization.
- ABC-aware storage assignment support.
- CSV import/export support.

12.3 Skills Learned

- Relational database modelling and normalization.
- Primary/foreign-key relationship design.
- SQL joins, aggregates, views, stored procedures and triggers.
- REST API development with FastAPI.
- JWT authentication and user-scoped access.
- Transaction-safe inventory operations.
- Algorithm implementation using real application data.
- Shortest-path and heuristic route optimization.

- Frontend–backend integration.
- Debugging, testing and technical documentation.

Overall, the project provides a strong academic example of how database engineering, web development and algorithmic decision support can be combined to solve a practical warehouse-management problem.

CHAPTER -13 REFERENCES

1. Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, Pearson.
2. Silberschatz, A., Korth, H. F. and Sudarshan, S., Database System Concepts, McGraw-Hill.
3. MySQL 8.0 Reference Manual, Oracle Corporation.
4. FastAPI Documentation, FastAPI official documentation.
5. Python Documentation, Python Software Foundation.
6. Uvicorn Documentation, Uvicorn official documentation.
7. Bootstrap Documentation, Bootstrap official documentation.
8. JSON Web Token (JWT) documentation and standards references.
9. Dijkstra, E. W., shortest-path algorithm concepts and graph algorithms references.
10. Project source code, SQL scripts, API implementation and test records used during development.

CHAPTER -14 APPENDICES

Appendix A – Application Screenshots

Insert the final screenshots captured from the working application in the following order. The screenshots should be taken from the fixed/current project version so that the documentation and implementation remain consistent.



Figure A1 – Login / Authentication Screen

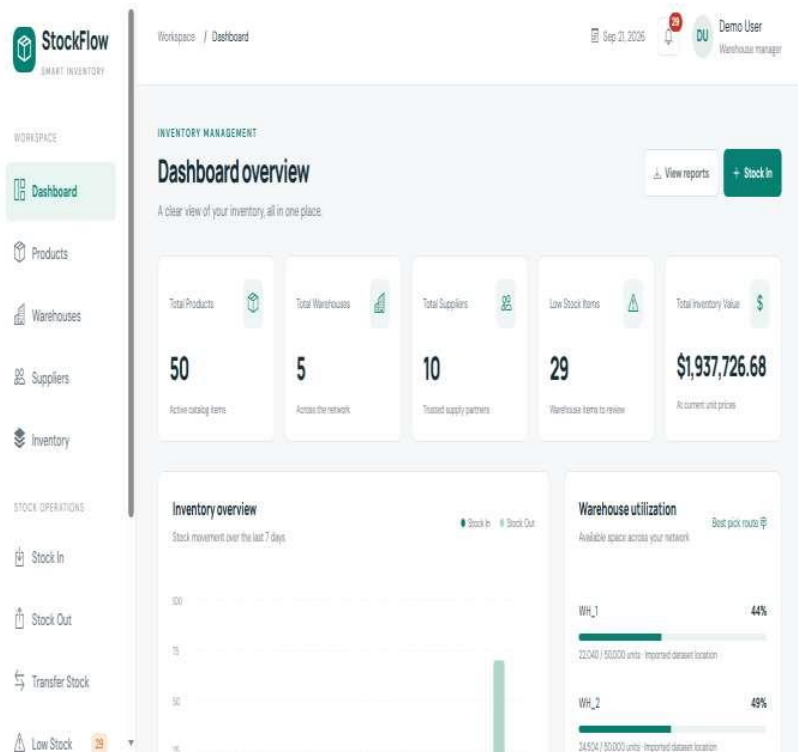


Figure A2 – Main Dashboard

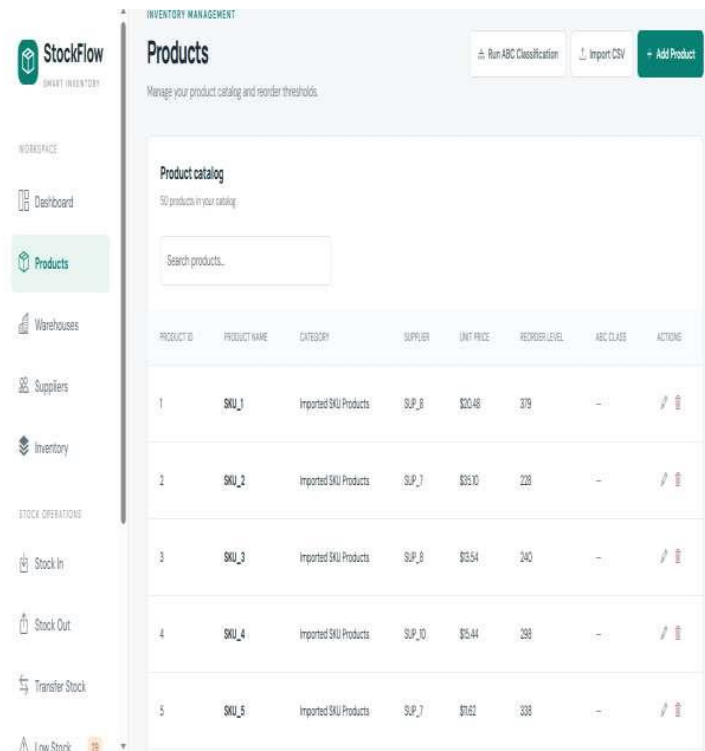


Figure A3 – Product Management

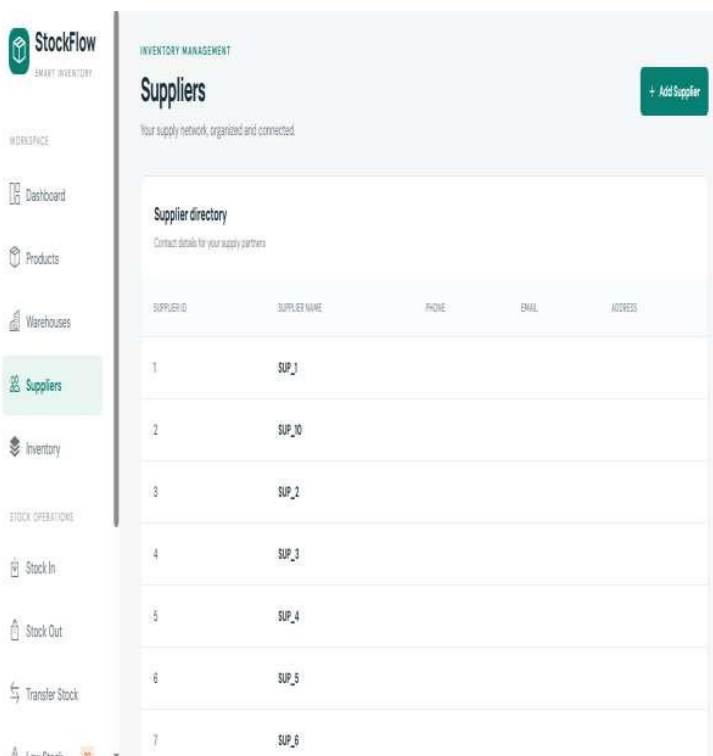


Figure A4 – Supplier Management

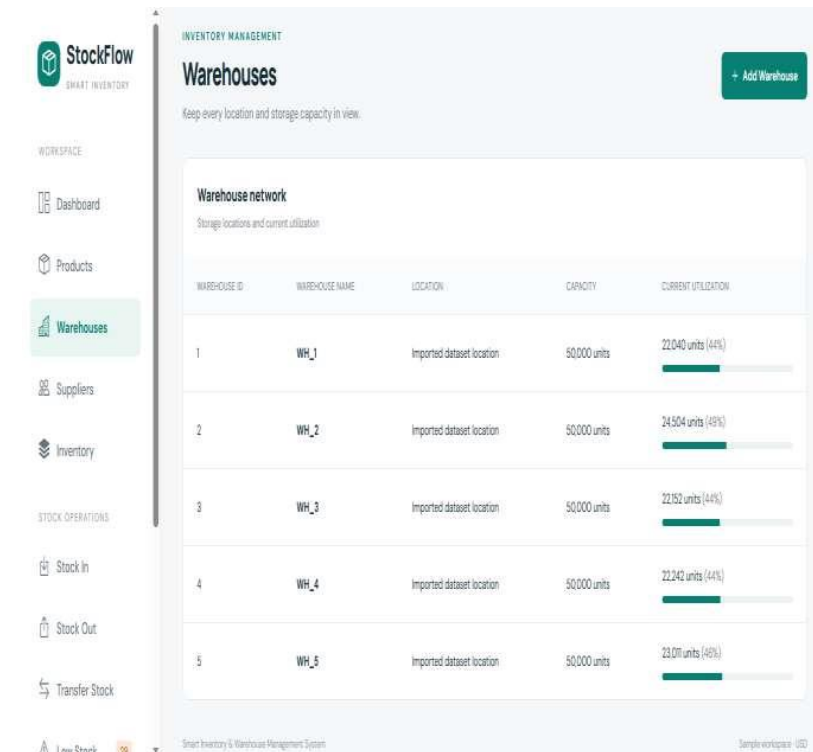


Figure A5 – Warehouse Management

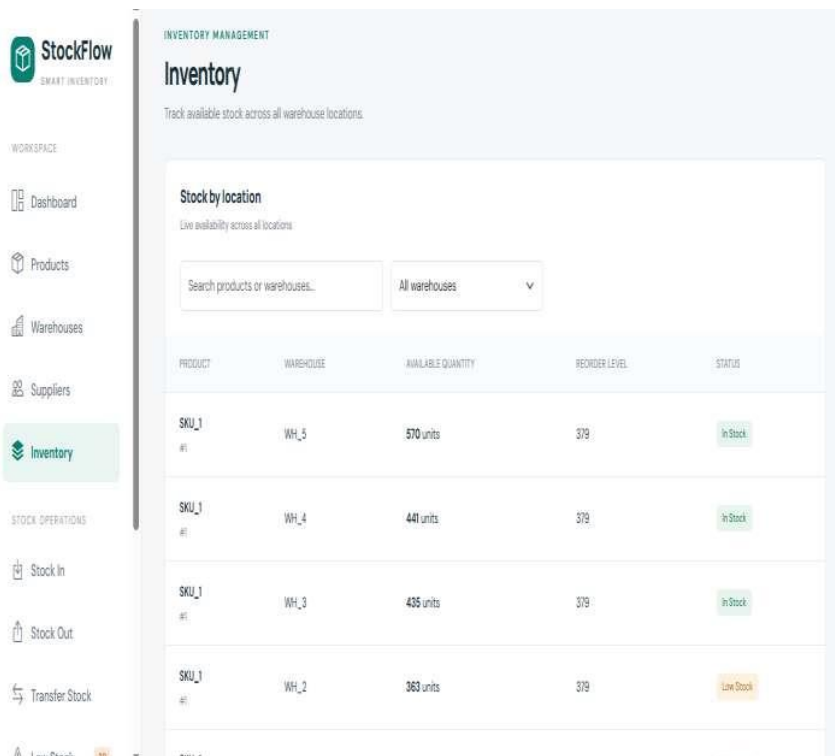


Figure A6 – Stock / Inventory View

Figure A7 – Stock-In / Stock-Out / Transfer

StockFlow

SMART INVENTORY

WORKSPACE

Dashboard

Products

Warehouses

Suppliers

Inventory

STOCK OPERATIONS

Stock In

Stock Out

Transfer Stock

INVENTORY MANAGEMENT

Stock In

Receive products and update your warehouse inventory.

Stock receipt details

Fields marked by the form are required.

Product

SKU_1

Warehouse

WH_1

Reason

Customer order

Quantity

Enter units

Date

09/21/2026

Reset

Record Stock

Stock snapshot

Check availability before you proceed

SKU_1

#1 Imported SKU Products

554 units available

WH_1

378 units

Reorder level

\$20.48

Unit price

Each saved movement updates inventory and the transaction history.

(a) Stock-In

StockFlow

SMART INVENTORY

WORKSPACE

Dashboard

Products

Warehouses

Suppliers

Inventory

STOCK OPERATIONS

Stock In

Stock Out

Transfer Stock

INVENTORY MANAGEMENT

Stock Out

Record dispatches, consumption, and stock adjustments.

Stock dispatch details

Fields marked by the form are required.

Product

SKU_1

Warehouse

WH_1

Reason

Customer order

Quantity

Enter units

Date

09/21/2026

Reset

Record Stock Out

Stock snapshot

Check availability before you proceed

SKU_1

#1 Imported SKU Products

554 units available

WH_1

378 units

Reorder level

\$20.48

Unit price

Each saved movement updates inventory and the transaction history.

(b) Stock-Out

WORKSPACE

Dashboard

Products

Warehouses

Suppliers

Inventory

STOCK OPERATIONS

Stock In

Stock Out

Transfer Stock

Transfer Stock

Move inventory seamlessly between your warehouses.

Transfer details

Fields marked by the form are required.

Product

SKU_1

From Warehouse

WH_1

To Warehouse

WH_2

Quantity

Enter units

Date

09/21/2025

Reset

Confirm Transfer

Stock snapshot

Check availability before you proceed

SKU_1

#1 Imported SKU Products

554 units available

WH_1

379 units

Reorder level

\$20.48

Unit price

363 units at destination

WH_2

Each saved movement updates inventory and the transaction history.

(c) Transfer Stock

WORKSPACE

Dashboard

Products

Warehouses

Suppliers

Inventory

STOCK OPERATIONS

Stock In

Stock Out

Transfer Stock

Transaction History

Transactions

A complete record of your inventory movements.

Transaction history

29491 recorded stock movements.

Search transactions...

All transaction types

TRANSACTION ID	PRODUCT	TYPE	QUANTITY	WAREHOUSE	DATE
29491	SKU_1	OUT	20	WH_1	2025-09-21
29490	SKU_1	OUT	50	WH_1	2025-09-21
29489	SKU_50	OUT	15	WH_5	2024-12-30
29488	SKU_50	OUT	24	WH_5	2024-12-29
29487	SKU_50	OUT	17	WH_5	2024-12-28
29486	SKU_50	OUT	21	WH_5	2024-12-27

Figure A8 – Transaction History

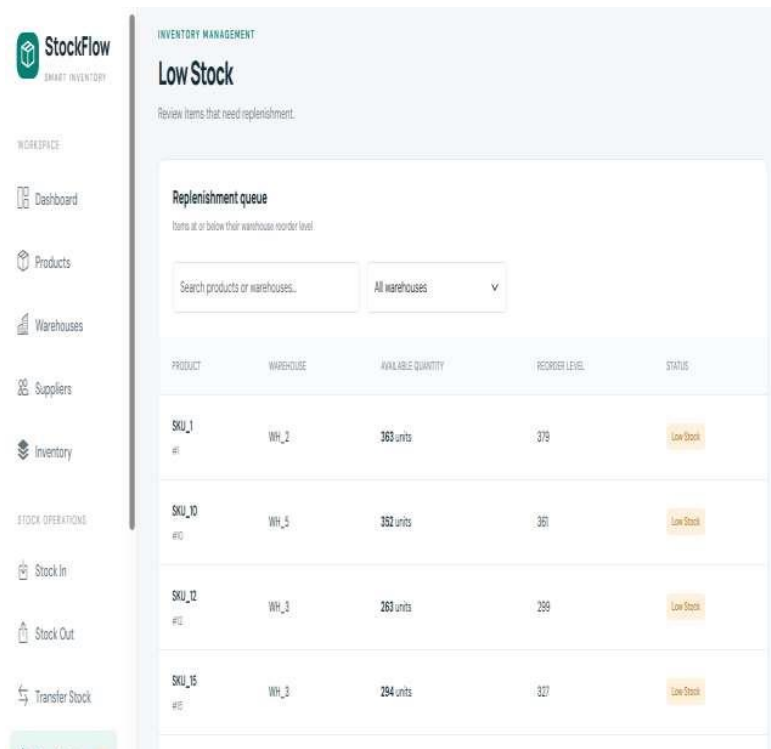
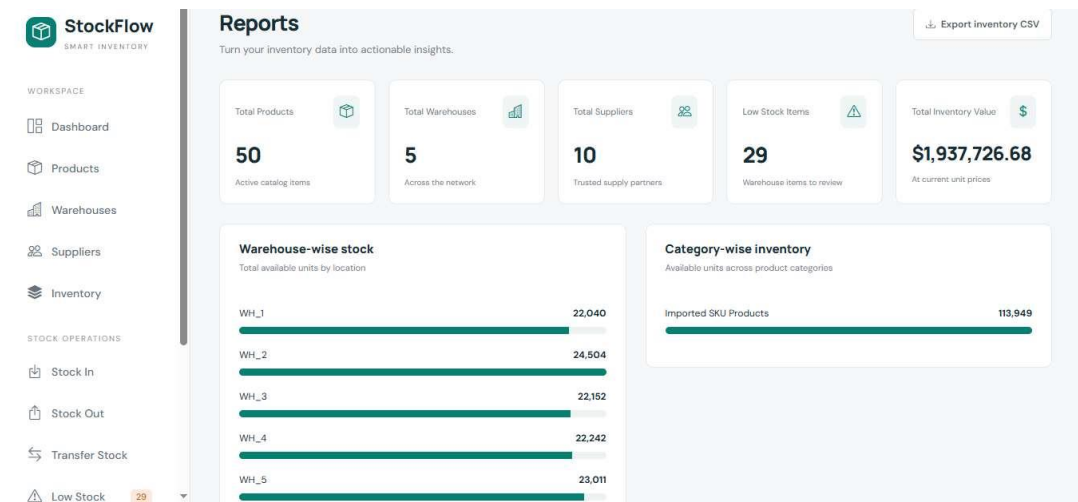


Figure A9 – Reorder Alerts

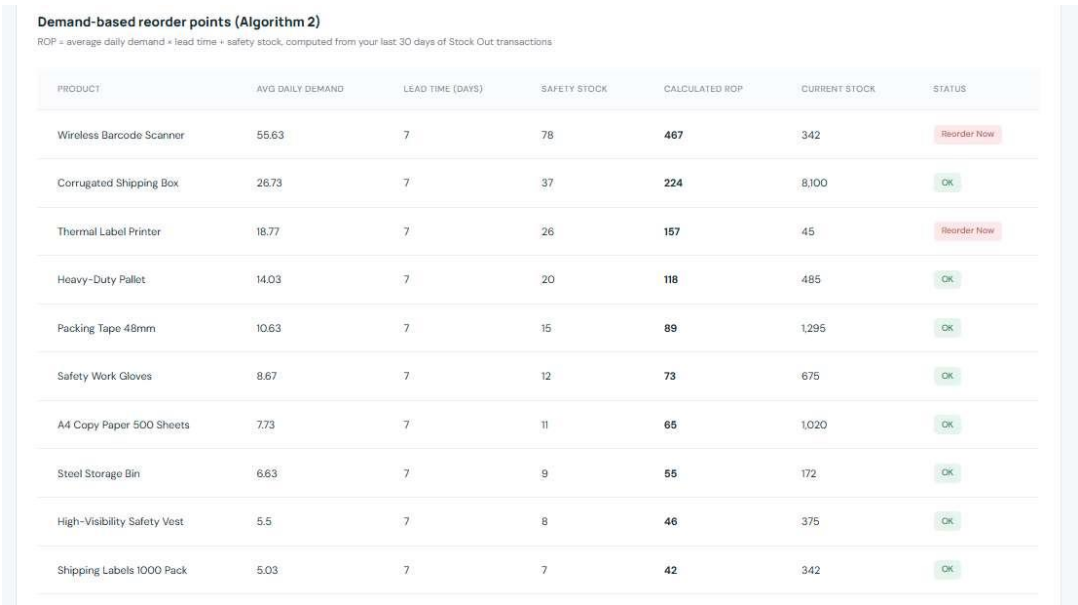
[SCREENSHOT PLACEHOLDER]

Figure A10 – ABC Classification Results

Figure A11 – Reorder-Point / Analytics Result



(a) Reports and Analytics Summary



(b) Demand-Based Reorder Points



Figure A12 – Warehouse Storage Locations

Warehouse picking-route optimizer (Algorithm 3 – Dijkstra)

Pick a warehouse and a few products; the shortest walking route between their storage bins is computed with Dijkstra's algorithm

Warehouse

Central Distribution

Products to pick (ctrl/cmd-click for multiple)

Wireless Barcode Scanner
Corrugated Shipping Box
Thermal Label Printer
Heavy-Duty Pallet
Parking Tane /18mm

Optimize Route

Optimized route: ENTRANCE → A-01 → A-02 → A-03 → A-02 → A-01 → ENTRANCE → B-01

Optimized distance

36.98 m

Naive (given order)

36.98 m

Route reduction

0%

Figure A13 – Dijkstra Picking Route

[SCREENSHOT PLACEHOLDER]

Figure A14 – CSV Import / Export

[SCREENSHOT PLACEHOLDER]

Figure A15 – API Documentation / Endpoint Verification

Appendix B – Representative API Endpoints

Area	Representative Endpoint / Purpose
Authentication	Login / token generation
Products	Product creation, retrieval and update
Inventory	Current stock and inventory summaries
Transactions	Stock movement and transaction history

Area	Representative Endpoint / Purpose
Analytics	ABC classification and inventory analytics
Reorder Points	30-day demand-based reorder calculation
Alerts	Reorder-alert retrieval
Routes	Warehouse shortest-path / picking route
CSV	Bulk product import and inventory export

Appendix C – Key Formulas

Consumption Value = Total OUT Quantity × Unit Price

ROP = Average Daily Demand × Lead Time + Safety Stock

ABC:

A → cumulative consumption value ≤ 70%

B → cumulative consumption value > 70% and ≤ 90%

C → remaining products

Appendix D – User Workflow

1. Log in using a registered account.
2. Create or import product information.
3. Add categories, suppliers and warehouses as required.
4. Record incoming stock.
5. Record stock-out or warehouse transfers.
6. Monitor current stock and transaction history.
7. Check reorder alerts.
8. Run ABC and reorder-point analytics.
9. Use warehouse route optimization for picking.
10. Export inventory information when required.

Appendix E – Documentation Accuracy Note

This report intentionally distinguishes implemented functionality from future enhancements. In particular, ABC classification is implemented in the FastAPI/Python analytics layer using 70% and 90% cumulative-value thresholds; the reorder-point analysis uses a 30-day OUT-transaction window; and warehouse routing uses Dijkstra shortest paths together with a nearest-

neighbour stop-selection strategy. The system identifies replenishment requirements but does not claim to automatically determine purchase quantity or execute procurement.